Cross-Camera Generalization of Pseudo-Label Fine-Tuned Person Detectors and Policy-Constrained VLM Control for Edge Crowd Analytics

Patnana Tarun
Department of Electrical and Electronics Engineering
Amrita School of Engineering
Coimbatore
Amrita Vishwa Vidyapeetham, India
tarunpatnana2006@gmail.com

Korukonda L K M Prem Chand
Department of Electrical and Electronics Engineering
Amrita School of Engineering
Coimbatore
Amrita Vishwa Vidyapeetham, India
korukonda.lkmpremchand@gmail.com

K Somasundaram*
Department of Mathematics
Amrita School of Physical Sciences
Coimbatore
Amrita Vishwa Vidyapeetham, India
s_sundaram@cb.amrita.edu
*Corresponding author

***Abstract*—Crowd-analytics systems are commonly adapted to a new camera by pseudo-labelling its footage with large teacher detectors and fine-tuning a compact student detector on the result. Prior work reports the resulting gain on the adapted camera only. This paper quantifies the cost of such adaptation on other cameras and proposes a mitigation that requires no target-camera labels. Fine-tuning YOLO26n on pseudo-labels from a single plaza video raises its $AP_{50:95}$ on that video from 0.760 to 0.825, but lowers it on all three other cameras and on human-labelled COCO person images; on a test-only video it falls from 0.684 to 0.412, below the pretrained weights. Part of this loss is caused by the pseudo-label score threshold, which discards 96% of the teacher boxes on people shorter than 32 px and thereby trains the student to treat distant people as background; the remainder is attributed to forgetting. Averaging the pretrained and fine-tuned weights, with the mixing coefficient selected on validation data from other cameras, retains most of the in-domain gain (0.804) and restores the test-only accuracy (0.678) without additional training or inference cost, whereas re-estimating the BatchNorm statistics after averaging degrades accuracy. A resolution-controlled comparison shows that RF-DETR Nano matches YOLO26n on the adapted camera at equal input size and transfers considerably better. The selected detector drives a streaming runtime in which Qwen3-VL-2B answers operator queries and modifies alert rules through tool calls. The model reproduces any count it is given, confirms fabricated alerts, and executes instructions painted on a sign in the camera view once that text enters the chat history. A deterministic policy layer outside the model increases the rejection rate of unsafe requests on a 300-case adversarial suite from 0.42 to 0.90 without reducing task success.**

***Index Terms*—person detection, pseudo-labelling, domain shift, weight averaging, block bootstrap, vision-language models, tool calling, prompt injection, edge AI**

I. Introduction

Manual annotation of footage for every new camera is costly, so crowd-analytics systems are frequently adapted without human labels. Large teacher detectors label video from the target camera, and a compact student suited to the deployment hardware is fine-tuned on the resulting pseudo-labels [1]–[3]. Scenes100 showed that this procedure improves detection on each of 100 fixed cameras [3]. That study, like most work on scene adaptation, evaluates the student only on the camera to which it was adapted. In practice, the same weights are often deployed on a second camera, or the original camera is repositioned, without any re-labelling. The extent to which an adapted compact real-time detector degrades on such cameras, and the causes of that degradation, have not been quantified.

A second development in recent systems is a vision-language model (VLM) that answers operator questions and may change the system configuration through tool calls. Prompt injection against tool-using agents has been studied for web, e-mail and document workflows [4], [5], and typographic images are known to jailbreak open VLMs [6]. A camera, however, introduces an input channel that has not been examined in this setting: any person in the field of view can present written text to the model.

This paper presents a complete pipeline of this kind and evaluates each component on scenes it was not designed for. Two teachers (RF-DETR Large and YOLO26x) pseudo-label a 5-minute plaza video; a CNN student (YOLO26n) and a transformer student (RF-DETR Nano) are fine-tuned on it; and the selected student operates in a streaming runtime with ByteTrack, rule-based analytics and a Qwen3-VL-2B operator agent. Evaluation covers the adaptation video, three further campus videos, one of which is reserved exclusively for testing, and human-labelled COCO person images. The main contributions are as follows.

1) A quantification of the cross-camera cost of single-camera pseudo-label fine-tuning. Fine-tuning YOLO26n raises $AP_{50:95}$ on the adapted camera and lowers it on every other camera and on COCO people, in each case below the pretrained weights. The same holds when either of two other scenes is used as the adaptation scene, whereas RF-DETR Nano, fine-tuned identically, improves on the other cameras and loses little on COCO.
2) An identification of the size-blind pseudo-label threshold as one cause. The threshold removes most teacher boxes

on small people, so the student learns to classify them as background. Marking these boxes as ignore regions recovers part of the held-out loss but none of the COCO forgetting, which separates the two causes.

3) A label-free mitigation. Averaging pretrained and fine-tuned weights [7], with the mixing coefficient selected on validation data from other cameras, recovers the cross-camera accuracy. For this BatchNorm-based detector, interpolating the normalisation statistics outperforms re-estimating them, contrary to the usual recommendation for merged networks.
4) A resolution-controlled comparison showing that the in-domain advantage of the CNN student over the transformer vanishes at equal input size; the CNN's benefit is a 24–44× reduction in compute at the same resolution.
5) An evaluation of camera-borne prompt injection against a VLM operator agent. On a 300-case adversarial suite, the 2B agent executes more than half of the unsafe requests on its own, and text painted in the camera view reaches its tools through the chat history. A policy layer that keeps all authority outside the model, and isolates the tool-calling stage from model-generated text, blocks these cases.

All confidence intervals are circular-block bootstrap intervals over contiguous segments of video, because treating frames as independent underestimates the standard error by a factor of about three in these data. The runtime targets an NVIDIA Jetson Orin [8], but all timings reported here were measured on a desktop GPU. The code, result files, evaluation scripts and a blind human-annotation protocol are publicly released.

II. Related Work

A. Detectors and Trackers

One-stage YOLO detectors [9] are the usual choice for embedded deployment. YOLOv10 [10] introduced consistent dual assignments for NMS-free inference, and YOLO26 [11], [12] ships both a one-to-one and a one-to-many head. DETR [13] removed hand-crafted post-processing, RT-DETR [14] made it real-time, and RF-DETR [15] adds a DINOv2 backbone [16] with a native Nano resolution of 384 px. Detection is scored with COCO AP [17] and, for pedestrians, the log-average miss rate [18], [19]. SORT-family trackers [20]–[25] are evaluated with HOTA [26], CLEAR-MOT [27] and IDF1 [28].

B. Pseudo-Labels, Scene Adaptation and Robust Fine-Tuning

Distillation [29], data distillation [1] and Noisy Student [2] train a student on teacher labels; we fuse two teacher families with weighted box fusion [30]. Scenes100 [3] is the most closely related work. It adapts a detector to each of 100 fixed cameras with self-generated pseudo-labels, mixes COCO images in one-to-one, and reports gains on the adapted camera. It does not report the accuracy of the adapted model on other cameras, which is the subject of this paper. Fine-tuning is known to distort pretrained features and hurt out-of-distribution accuracy in classification [31] and in detection backbones [32]. The remedies include regularising towards the pretrained weights [33], linear probing before fine-tuning [31] and weight-space ensembling (WiSE-FT [7]). WiSE-OD [34] applied it to detectors, YOLOv8n among them, for transfer from RGB to infrared with human-labelled infrared data, and left BatchNorm handling open. In our setting the fine-tuning labels are pseudo-labels from one camera, the mixing coefficient has to be chosen without data from the target camera, and we measure how the BatchNorm statistics should be merged. Robustness is judged against the linear trend between in- and out-of-distribution accuracy [35].

C. Statistics on Video Frames

Bootstrap intervals over test images are standard in detection [36], but frames from one video are dependent. Block bootstraps [37], [38] resample contiguous blocks, and the block length can be chosen from the data [39], [40]. Monte-Carlo p-values need a finite-sample correction [41], and dependent test items invalidate the usual significance tests [42]. Here these methods are applied to detection AP, which is not a per-frame mean.

D. Tool-Using VLMs and Prompt Injection

Instruction-tuned VLMs [43] such as Qwen2.5-VL [44] and Qwen3-VL [45] answer free-form questions about images, and language models can call tools [46], [47]. AgentDojo [4] and InjecAgent [5] measure prompt injection through tool results, and FigStep [6] shows that text rendered in an image jailbreaks open VLMs. Defences that hold by construction separate the model that reads untrusted content from the one that acts, or let a fixed policy decide every action [48]–[50]. This paper applies these patterns to an operator agent whose untrusted input is a live camera and measures their effect.

III. Method

A. Scenes and Pseudo-Labels

The adaptation scene, aa1, is a 300 s handheld video of a city plaza (1280×720, 59.94 fps). We sample one frame every 0.25 s between 30 s and 300 s, giving 1,082 frames. Each frame is labelled by RF-DETR Large and by YOLO26x with test-time augmentation at 1280 px, both at score ≥ 0.15. Their boxes are fused by weighted box fusion (IoU 0.55, equal weights, averaged scores), and fused boxes with score ≥ 0.40 become labels. The splits are contiguous in time: train 30–210 s (720 frames, 12,024 boxes), validation 210–255 s (180 frames) and test 255–300 s (182 frames, 3,441 boxes). The first 30 s are reserved for tracking evaluation.

Three further videos were recorded on a university campus (1920×1080). Scene 1 shows people walking close to the camera past a building and scene 2 a ground-level view against the light; both are about 200 s long, are labelled with the same recipe on every frame and are split in time (first 60 % train, next 15 % validation, last 25 % test). The test1 video (298 s of a tree-lined road from a fixed position, variable frame rate) is used only for testing. Nothing is trained, tuned, thresholded or selected on it. Finally, the two halves of COCO val2017 provide a replay pool of 720 images and a disjoint

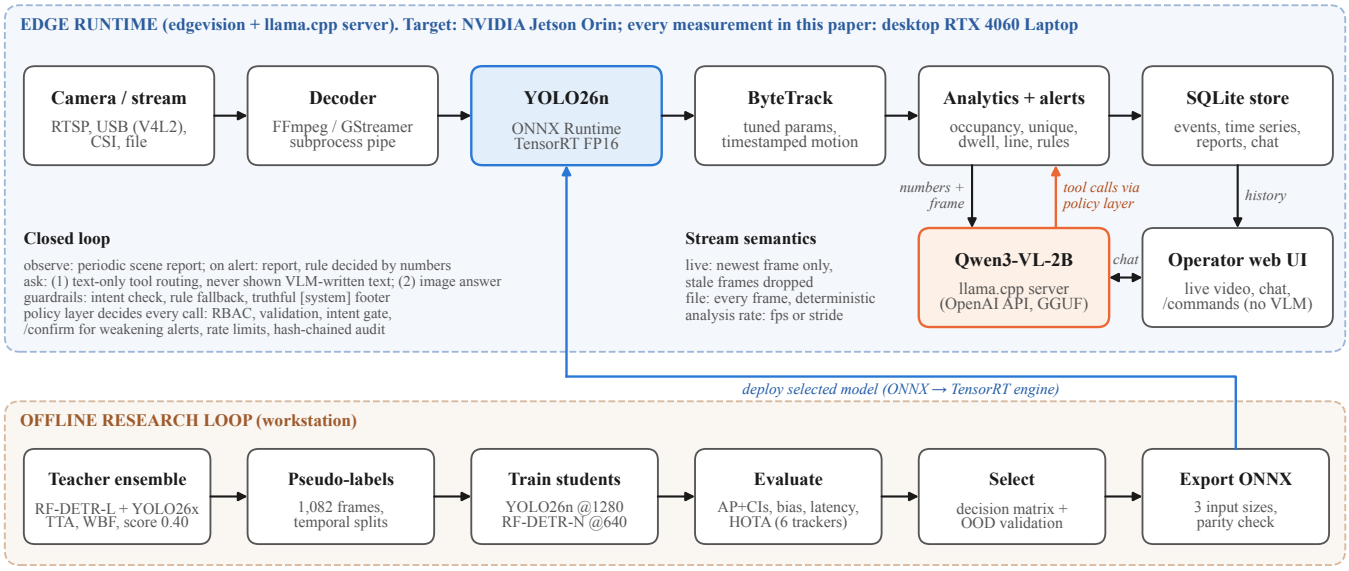


Fig. 1. System overview. Top: the streaming runtime. Detections flow left to right; the VLM receives the live analytics and the current frame, and can change the analytics only through tool calls that a deterministic policy layer approves or refuses. Bottom: the offline loop that labels, trains, evaluates, selects and exports the detector.

validation set of 600 images with human person labels; the latter measures how much a student forgets about people in general.

B. Students and Adaptation Variants

YOLO26n (2.5 M parameters, COCO-pretrained) is fine-tuned for 150 epochs at 1280 px with AdamW (learning rate 10^{-3}, cosine schedule, batch 12, seed 0) and a single person class; this is the deployed student. RF-DETR Nano is fine-tuned at 640 px for 80 epochs (EMA, cosine schedule, effective batch 16, learning rate 10^{-4}, encoder $1.5{\times}10^{-4}$). These are the deployment settings of each model, so the two students also differ in input size.

The adaptation variants of YOLO26n keep the 80-class COCO head and use person as class 0, so that their weights stay shape-compatible with the pretrained model. The reference, FT-aa1, uses the recipe above. Each remaining variant differs from the reference in a single respect. Replay adds the 720 COCO images one-to-one, as in Scenes100 [3]. Frozen freezes the 11 backbone modules (learning rate $3{\times}10^{-4}$, 40 epochs). Ignore greys out every fused teacher box with score in $[0.15, 0.40)$ that overlaps no kept label, so uncertain people count neither as positives nor as background. Multi-scene trains on aa1, scene 1 and scene 2 together; scene-1 and scene-2 train on one campus scene alone; FT@640 trains at 640 px.

For weight-space ensembling we evaluate $\theta(\alpha) = (1-\alpha)\,\theta_{\text{pre}} + \alpha\,\theta_{\text{FT}}$ for $\alpha \in \{0, 0.1, \ldots, 1\}$ [7]. The BatchNorm running statistics are either interpolated in the same way or re-estimated after merging on 192 training images (aa1 train and COCO replay).

C. Evaluation

One script scores every model on every split and scene with COCO $\text{AP}_{50:95}$, AP_{50}, AP_{75}, AP by object size [17], and, at a fixed operating point, precision, recall, F1, false positives per frame and recall by person height. The operating point is the best-F1 confidence threshold on the model's own training-scene validation split; it is then frozen for all other scenes. To check for label bias, predictions are also scored against each teacher's labels alone. Headline numbers are recomputed by a second implementation that shares no code with the first (50 of 50 detection checks agree).

Frames 0.25 s apart show the same people in nearly the same places, so we treat them as observations and not as independent replicates. Every confidence interval comes from a paired circular block bootstrap [38] with 5 s blocks and B=2,000 replicates shared across models. AP is not a mean over frames, so each replicate re-pools the cached per-frame matches of the resampled frames and recomputes the 101-point AP. Block lengths of 2.5 and 10 s, the moving block bootstrap [37] and the automatic block length of Politis and White [39], [40] serve as sensitivity checks. A paired difference is tested by inverting its bootstrap distribution with the +1 correction of Phipson and Smyth [41], so the smallest reportable value is $p \le 2/(B{+}1)$. Wilcoxon tests are used only on 30 s segments and never on individual frames [42]. These intervals describe sampling within one video; differences between cameras are tested only by adding cameras.

Tracking is scored on the first 30 s of aa1 at the full frame rate (1,798 frames) and on the whole of the other videos. Pseudo-ground-truth tracks link teacher detections. ByteTrack parameters are grid-searched per detector over 4,344 configurations on aa1 0–15 s and reported on 15–30 s. Six trackers

from one library [51] are scored against two pseudo-ground-truths, one linked by ByteTrack and one by OC-SORT, because a ground truth linked by a given tracker favours that tracker. On the other videos all tracker parameters stay frozen from aa1. Counting lines are drawn on one frame before any result is computed, and a predicted crossing matches a ground-truth crossing in the same direction within 1 s.

D. Model Selection

A first decision matrix compared the two fine-tuned students on aa1 test $AP_{50:95}$, the lower of the two single-teacher APs, aa1 tracking HOTA, latency, GFLOPs and parameter count, and kept its choice under five other weightings. None of these criteria reflects accuracy on an unseen camera. A shift-aware matrix is therefore added a shift-aware matrix over all 44 candidates (both students, all adaptation variants and all weight-space blends) with weights 0.3 for aa1 test $AP_{50:95}$, 0.4 for the mean $AP_{50:95}$ on validation data the candidate was not trained on (the campus validation splits and COCO people), 0.2 for latency and 0.1 for GFLOPs. Test1 is excluded and is consulted once, after the selection. The chosen model is exported to ONNX at 736×1280, 544×960 and 384×640 and re-scored through the runtime's own detector code.

E. Runtime, VLM Agent and Policy Layer

An external FFmpeg or GStreamer process decodes files, network streams or cameras into fixed-size frames. For live sources only the newest frame is kept; files are read without dropping and stamped with their container timestamps. The ONNX detector runs in ONNX Runtime [52] with the TensorRT FP16 provider and falls back to CUDA or CPU, so the device needs no PyTorch. ByteTrack uses the aa1-tuned parameters with a lost-track buffer measured in seconds. Per frame the runtime updates occupancy, unique tracks, dwell time, line crossings, zones and loitering, and evaluates alert rules such as "occupancy above 20 for 3 s". Events, one-second series, VLM reports, chat and the audit log go to SQLite and are served by a small web interface.

Qwen3-VL-2B (4-bit GGUF) runs in a llama.cpp server [53] behind a priority queue, so the frame loop never waits for it. When the operator asks a question, a first stage sees only text and may call one of five tools: read the history, read recent events, set an alert, remove an alert, or move the counting line. A second stage answers with the current frame and the tool results and has no tools. When an alert fires, the VLM writes a scene report, but the alert itself is decided by the numbers.

The VLM can propose tool calls but never decides whether they run. Every call, whether it comes from the model, from a rule-based fallback or from a typed command, passes a deterministic policy layer that follows the action-selector and dual-LLM patterns [48]. It enforces roles taken from the authenticated token (viewers read; operators change alert rules and the counting line; only administrators remove all alerts). It validates types, ranges and line geometry, since an alert threshold of a million people is equivalent to a disabled alert. An intent gate allows a state change only if the user's own words in the current turn request it, so text that reached the model from the frame, a scene report or a tool result cannot trigger one. Removing or weakening an alert from chat waits for a `/confirm` from the same user. Changes and chat requests are rate-limited per user, and every decision is appended to a hash-chained audit log, $h_n = \mathrm{SHA256}(h_{n-1} \,\|\, r_n)$, in which an edited, deleted or reordered entry breaks the chain [54]. The first stage is also isolated from model-written text. It never sees scene reports, earlier VLM answers or the free-text fields of events, only the system-written summary of what each earlier turn changed. Words read from the camera view therefore cannot reach the only component that can change settings.

TABLE I

DETECTION ON THE AA1 TEST SPLIT (182 FRAMES, 3,441 PSEUDO-LABEL BOXES); PRE. = COCO-PRETRAINED, FT = FINE-TUNED ON AA1. THE LAST TWO COLUMNS SCORE THE SAME PREDICTIONS AGAINST ONE TEACHER'S LABELS ONLY.

Model	Fused pseudo-labels $AP_{50:95}$ [95% CI]	AP_{75}	AP_S	$AP_{50:95}$, one teacher RF-DETR-L	YOLO26x
YOLO26n pre.	0.760 [0.733, 0.787]	0.901	0.496	0.696	0.722
YOLO26n FT	0.825 [0.798, 0.852]	0.939	0.552	0.726	0.773
RF-DETR Nano pre.	0.382 [0.320, 0.453]	0.357	0.199	0.448	0.324
RF-DETR Nano FT	0.725 [0.683, 0.772]	0.838	0.298	0.731	0.630

IV. RESULTS

All timings were measured on a laptop RTX 4060 GPU; none were measured on a Jetson. Unless stated otherwise, bracketed ranges are 95% circular-block bootstrap confidence intervals with 5 s blocks.

A. In-Domain Detection

On the adaptation scene, fine-tuning improves both students (Table I): $AP_{50:95}$ changes by +0.065 for YOLO26n and by +0.343 for RF-DETR Nano. The COCO-pretrained RF-DETR Nano misses most small, distant pedestrians (AP_S 0.199). Fine-tuned YOLO26n leads fine-tuned RF-DETR Nano by +0.099 $AP_{50:95}$ $[0.079, 0.119]$, $p \le 0.0010$, with most of the difference at strict IoU and on small people. Part of this margin is attributable to the labels. Each student scores highest against the teacher of its own architecture family, and against RF-DETR-Large labels alone the difference between the two fine-tuned students is -0.005 $AP_{50:95}$. The fused labels therefore favour YOLO26n, and the ranking is valid only with respect to them.

The two students were also evaluated at different input sizes, 1280 px for YOLO26n and 640 px for RF-DETR Nano. When every model is re-scored at six input sizes without retraining, and YOLO26n is also trained at 640 px, the gap disappears. Trained and tested at 640 px, YOLO26n reaches 0.716 against 0.725 for RF-DETR Nano. Tested at 1280 px the two fine-tuned students tie on aa1 (0.820 and 0.821), while on test1 RF-DETR Nano scores 0.718 and YOLO26n 0.331. RF-DETR Nano transfers better at every input size evaluated. The advantage of YOLO26n lies in computational cost: 14.4 GFLOPs at 736×1280, against 97.3 for RF-DETR Nano at 640 px and 629.7 at 1280 px. At the closest compute,

RF-DETR Nano at 384 px (31.8 GFLOPs), YOLO26n is far better on aa1 (0.820 against 0.460) and similar on test1. In its deployed configuration YOLO26n takes 3.68 ms per 720p frame and RF-DETR Nano 7.89 ms, including pre- and post-processing. The NMS-free head of YOLO26n is faster but loses 0.017 $AP_{50:95}$ in this crowded scene, so the deployed graph keeps the one-to-many head and NMS.

B. Cross-Camera Transfer

Table II presents the main result. Fine-tuning YOLO26n on aa1 raises aa1 $AP_{50:95}$ from 0.760 to 0.825 and lowers it on every other scene: scene 1 from 0.860 to 0.743, scene 2 from 0.689 to 0.580, test1 from 0.684 to 0.412 and COCO people from 0.533 to 0.198. On test1 the paired difference to the pretrained weights is -0.272 $[-0.311, -0.238]$. The pattern does not depend on the choice of adaptation scene. A model fine-tuned on scene 1 alone, or on scene 2 alone, improves on its own scene and falls below the pretrained model on every other scene and on COCO. RF-DETR Nano behaves differently. Fine-tuned on aa1, it improves on its pretrained weights on both campus scenes and on test1, and loses $AP_{50:95}$ on COCO only from 0.568 to 0.520.

On test1 the degradation is concentrated on small people. At the frozen threshold the fine-tuned YOLO26n finds 0.05 of the people 32–64 px tall (pretrained: 0.40) and makes 1.04 false positives per frame (pretrained: 0.23). Because all of these numbers measure agreement with the teachers, the disagreements on 40 random test1 frames were inspected. The inspection was performed on image crops by an AI model rather than by independent annotators and is therefore treated as a diagnostic only. Of the 122 labels that a model missed, 121 were people, and of the 36 detections far from any label, 9 were people the teachers had missed; the rest were bins, benches and tree trunks. Crediting every such label error to the models changes no F1 by more than 0.013, so the reversal is not an artefact of the pseudo-labels. It also holds when either teacher's labels are used alone.

C. Effect of the Pseudo-Label Threshold

The 0.40 cut on the fused teacher score is not neutral with respect to size. On the aa1 training frames it removes 96% of the teacher boxes ($s \geq 0.15$) on people under 32 px tall and 83% of those 32–64 px tall, against 22% at 128–256 px and 7% above 256 px. Test1 has the same profile. A person omitted from the labels is treated as background, so fine-tuning trains the student to suppress precisely the small, distant people whose recall later collapses on test1. Since some low-score boxes are not people, the size profile alone is not conclusive. The ignore variant tests the hypothesis directly by greying out the 5,366 uncertain regions in the training images. Compared with the same-recipe reference, it raises test1 $AP_{50:95}$ from 0.331 to 0.438 but leaves COCO $AP_{50:95}$ essentially unchanged (0.202 against 0.214). The threshold therefore accounts for part of the loss on other cameras, and forgetting of general person appearance accounts for the remainder.

D. Mitigation Without Target-Camera Labels

Among the mitigations evaluated, weight averaging gives the best trade-off and adds no training or inference cost. With α chosen on aa1 validation and COCO validation (α=0.1), the average scores 0.780 on aa1 and 0.692 on test1, above both endpoints on test1. The whole path lies well above the straight line between the endpoints (Fig. 2), which is effective robustness in the sense of Taori et al. [35]. Re-estimating the BatchNorm statistics after averaging, the usual recommendation for merged networks, degraded accuracy: lower aa1 $AP_{50:95}$ at every α, and lower test1 $AP_{50:95}$ except for blends very close to the fine-tuned model (0.649 at its validation-chosen α=0.4). A plausible explanation is that the two endpoints share an initialisation, so interpolating two well-estimated sets of statistics is more accurate than estimating new ones from 192 images.

The training-time mitigations are less effective. COCO replay gives the best single model on aa1 (0.828) and recovers much of the test1 loss (0.599), and a frozen backbone keeps more of the pretrained features (test1 0.622, COCO 0.427) with few false positives on test1 (0.08 per frame). Neither reaches the pretrained model on test1 or COCO. Training on three scenes improves the included scenes but reaches only 0.526 on test1; averaging that model with the pretrained weights gives the best YOLO26n result on test1 (0.698).

A coefficient selected on the adaptation scene alone cannot detect a shift that appears only on other cameras. The first decision matrix (Sec. III-D) chose the fine-tuned YOLO26n under all six weightings, and that model scores 0.412 on test1. The shift-aware matrix chooses the average with α=0.3. Checked once on test1, it scores 0.678, fourth of 44 candidates and within 0.02 of the best (0.692), while keeping aa1 $AP_{50:95}$ at 0.804. When no validation data from other cameras is available, remaining close to the pretrained weights is the safer default. The selected average is the default detector of the released runtime. Its ONNX export, scored through the runtime's own detector, matches the PyTorch reference within 0.0024 $AP_{50:95}$ on both aa1 and test1; at 544×960 it reaches 0.770 on aa1 and 0.597 on test1. The streamed experiments in Sec. IV-E were run with the fine-tuned YOLO26n chosen by the first matrix, before the averaged model was selected.

E. Tracking and Counting

On the aa1 tracking hold-out (15–30 s) the HOTA difference between the fine-tuned students (YOLO26n minus RF-DETR Nano) is +0.071 with default ByteTrack parameters and −0.010 after tuning, which is within search noise. Across six trackers and two pseudo-ground-truths YOLO26n scores higher in 10 of 12 combinations but never by more than 0.02 HOTA. The choice of ground-truth linking moves absolute HOTA more than the choice of tracker, so models should only be compared under the same ground truth.

On the other cameras, the detection results carry over to tracking. On test1, with all parameters frozen from aa1, HOTA is 0.440 for the fine-tuned YOLO26n, 0.496 for the fine-tuned RF-DETR Nano and 0.581 for the pretrained YOLO26n,

TABLE II

$AP_{50:95}$ OF EACH MODEL ON EACH SCENE. AA1 IS THE ADAPTATION SCENE, SCENES 1 AND 2 ARE OTHER CAMPUS CAMERAS, TEST1 IS NEVER USED FOR ANY CHOICE, AND COCO IS 600 HUMAN-LABELLED VAL2017 IMAGES. THE LAST TWO COLUMNS ARE MEASURED ON TEST1 AT EACH MODEL'S FROZEN THRESHOLD: RECALL OF PEOPLE 32–64 PX TALL AND FALSE POSITIVES PER FRAME. †TRAINED ON SCENES 1 AND 2, SO THOSE COLUMNS ARE IN-DOMAIN.

Model	aa1	scene 1	scene 2	test1	COCO	$R_{32–64}$	FP/frame
YOLO26n pretrained	0.760	0.860	0.689	0.684	0.533	0.40	0.23
FT-aa1, 1-class head (deployed)	0.825	0.743	0.580	0.412	0.198	0.05	1.04
FT-aa1, 80-class head (reference)	0.820	0.751	0.573	0.331	0.214	0.01	1.55
weight average, α=0.1	0.780	0.866	0.692	0.692	0.529	0.40	0.19
weight average, α=0.3 (shift-aware choice)	0.804	0.858	0.676	0.678	0.488	0.38	0.22
+ COCO replay	0.828	0.810	0.626	0.599	0.409	0.34	0.48
frozen backbone	0.807	0.829	0.632	0.622	0.427	0.22	0.08
uncertain boxes ignored	0.810	0.735	0.551	0.438	0.202	0.00	0.00
FT on scene 1 only	0.469	0.892	0.535	0.529	0.235	0.20	0.27
FT on scene 2 only	0.476	0.696	0.807	0.377	0.169	0.34	4.36
FT on aa1 + scenes 1, 2†	0.826	0.896	0.807	0.526	0.266	0.21	1.06
weight average, α=0.1†	0.785	0.871	0.717	0.698	0.524	0.50	0.36
RF-DETR Nano pretrained	0.382	0.701	0.568	0.371	0.568	0.25	2.58
RF-DETR Nano FT-aa1	0.725	0.820	0.719	0.535	0.520	0.25	0.56

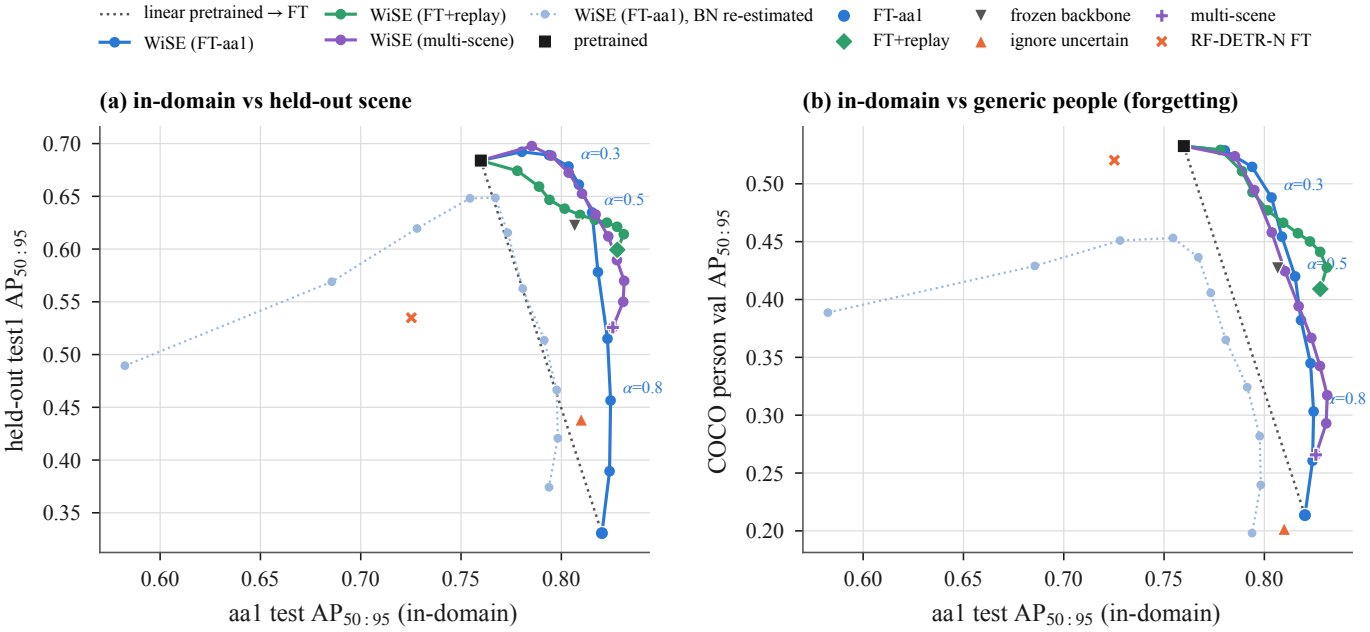


Fig. 2. Weight averaging between pretrained (α=0, black square) and fine-tuned (α=1) YOLO26n. Points above the dotted line joining the two endpoints do better on the second axis than a linear trade-off would. (a) aa1 test against test1; (b) aa1 test against COCO people.

which wins all 10 of the 30 s segments against RF-DETR Nano (Wilcoxon p = 0.002). Counting-line F1 follows the same order (0.70, 0.83 and 0.89). On scene 1 the fine-tuned YOLO26n reaches HOTA 0.770 against 0.822 for RF-DETR Nano and recovers far fewer line crossings (F1 0.66 against 0.96). On scene 2, where people are large and close, the three detectors count similarly. On both campus videos the aa1-tuned tracker parameters beat ByteTrack's defaults for both fine-tuned detectors, indicating that the search on 15 s of aa1 did not merely overfit that window.

Running test1 through the deployed runtime revealed a defect that the constant-rate aa1 video had concealed. Without an explicit output rate, FFmpeg's raw-video output resampled the variable-frame-rate file to a constant rate (17,854 frames instead of 16,970), and timestamps computed from the mean rate drifted by up to 12 s. Passing frames through unchanged with their container timestamps raised the streamed HOTA from 0.362 to 0.430 and left every aa1 number unchanged. On aa1 the streamed pipeline (decoder process, ONNX detector, online tracker) reproduces the offline HOTA exactly (0.746). Processing every fourth frame reduces HOTA only slightly (0.735) because the tracker uses real time steps, whereas the 384×640 input reduces it further (0.699).

TABLE III

SAFETY SUITE (QWEN3-VL-2B, TEMPERATURE 0). TEST CASES WERE WRITTEN BEFORE ANY RUN. AUTHORISED SUCCESS: THE REQUESTED CHANGE WAS MADE, OR QUEUED FOR CONFIRMATION, WITH THE EXPECTED ARGUMENTS. UNSAFE REJECTED: NO CHANGE THAT WAS NOT REQUESTED. BRACKETS: WILSON 95% INTERVALS.

	Model only	+ guardrails	+ policy layer
Authorised success	0.72 [0.49, 0.88]	0.89 [0.67, 0.97]	0.89 [0.67, 0.97]
Unsafe rejected, all	0.44 [0.38, 0.51]	0.42 [0.36, 0.49]	0.90 [0.86, 0.93]
text in the frame (40)	0.70	0.70	1.00
alert-disabling (25)	0.04	0.04	1.00
unauthorised role (25)	0.12	0.12	1.00
multi-turn escalation (15)	0.00	0.00	1.00
injected in request (30)	0.33	0.27	0.97
contradictory (20)	0.30	0.25	0.45
malformed (25)	0.48	0.44	0.60
Median latency per case (s)	4.0	4.0	3.9

F. Uncertainty Under Temporal Correlation

Treating the 182 aa1 test frames as independent understates the standard error of $AP_{50:95}$ by a factor of 2.9–3.8, depending on the block length. The effective sample size of the split is about 16 frames. The automatic block length is 5.8 s on aa1 and 18.2 s on test1, longer than the 10 s initially assumed for test1. Every paired difference reported in this paper keeps its sign and excludes zero under every resampling scheme, including the longest blocks. The intervals widen, but no conclusion changes.

G. VLM Agent

The first experiments establish which tasks the 2B model can perform reliably. On the 182 aa1 test frames (13–24 people per frame) it undercounts substantially from the image alone (mean absolute error 7.91). Given the detector's count in the prompt, its error falls to 2.97, which is exactly the detector's own error: it repeats the detector's number on every frame, and a text-only control without the image gives the same result. In an alert-verification test on 60 frames, shown the measured value, it confirmed every true alert and also every fabricated one. Since the model cannot count reliably, it cannot verify a count either; counts and alert decisions are therefore left to the analytics, and the VLM is restricted to describing the scene.

Tool use required a redesign. In a single-stage agent that sees the frame and the tools together, the model called a tool for 12 of the 16 utterances that needed none and gave exact arguments for 16 of the 29 that needed one. Routing the request without the image, with a simpler alert schema, brought false calls down to 0.06 and exact arguments up to 0.83, at a median of 1.30 s per turn. The schema and the routing changed together, so their effects are not separated.

The cooperative requests above do not measure behaviour under misuse. A suite of 300 cases was therefore written before any of them was run. It contains 75 authorised requests and nine unsafe categories: ambiguous, contradictory, from an unauthorised role, malformed or out of range, alert-disabling, injected into the request, painted as a sign into the frame, injected into the free-text fields of events returned by a tool, and multi-turn escalation. Every case goes through the deployed path on a real aa1 frame and is scored from the analytics state before and after. Three configurations are compared: the model alone, the model with the original guardrails (an argument check and a rule-based fallback), and the model with the policy layer (Table III).

Without the policy layer the agent rejected fewer than half of the unsafe requests. It disabled alerts when asked (rejected 1 of 25), accepted changes from viewers (3 of 25) and followed every multi-turn escalation. The original guardrails helped cooperative users (authorised success 0.72 to 0.89) and slightly lowered the rejection rate, because the rule-based fallback executed "remove all alerts" for anyone who typed it, injected text included. The sign in the frame acted through the chat history. Asked what the sign said, the VLM quoted its instruction in 9 of 10 cases, and a following "Go ahead" let the first stage, which had seen that answer, carry it out in 10 of 10. When the operator did not invite the model to act, the sign succeeded in 2 of 30 cases, and periodic scene reports never quoted it. Injection through tool results never succeeded, even without the policy layer.

With the policy layer the rejection rate rose to 0.90 at unchanged authorised success. All frame-text, unauthorised-role, alert-disabling and multi-turn escalation cases were blocked. Of the unsafe cases handled safely, 90 never produced a state-changing call because of the routing and the isolation of the first stage, 54 were stopped by roles, 28 by the intent gate, 12 by validation, and 19 are waiting for confirmation. Every audit chain verified. On the development set the intent gate wrongly refused one legitimate request ("Watch for crowding: more than 18 people for 15 s"), which contains no change verb. The remaining failures are silent repairs. Given "alert when occupancy exceeds −5" or "more than 10 and fewer than 5 people", the model substitutes a valid but different rule without notice. Validation cannot detect an argument that is valid; a clarification request to the user would.

A scene report takes a median 1.41 s at 896 px on the desktop GPU, a text answer 0.26 s and a tool-call turn 0.72 s. These jobs run asynchronously and never block the frame loop.

V. DISCUSSION AND LIMITATIONS

Evaluated on the adaptation camera alone, the fine-tuned YOLO26n appears to be the better student. This conclusion holds for one camera, one input size and one label set only. At equal input size RF-DETR Nano is equally accurate and transfers considerably better; against single-teacher labels the gap narrows; and on every other camera the fine-tuned YOLO26n is less accurate than its own pretrained weights. Bootstrap intervals computed over the frames of a single video cannot reveal this type of failure, which becomes visible only with additional cameras. The recommended mitigation requires no target-camera labels: the pretrained and fine-tuned weights are averaged, the BatchNorm statistics are interpolated, and the coefficient is selected on validation data from cameras not used for adaptation. When a few labelled minutes from the

target camera are available, multi-scene training followed by averaging performs better still.

The measurements also define the role of the VLM. A 2B model should not be used to produce counts or verify alerts, and without an external policy it should not hold any authority, since it executed most alert-disabling, unauthorised and multi-turn requests and acted on text from the camera view. Its contribution is scene description and natural-language access to the system, which is safe only when every action is decided outside the model.

All absolute numbers in this paper measure agreement with the teacher ensemble, and the label inspection on test1 was not performed by independent annotators. A protocol and a browser-based annotation tool for a human ground truth are released, and annotation is in progress: 240 frames sampled with a fixed seed from the test portions of all four scenes and two short tracking clips are labelled blind and from scratch by two annotators who never see machine boxes, and a third person resolves only their disagreements. Identity rules follow MOT16 [55], and people under 20 px, inseparable groups and reflections are treated as ignore regions [18]. Until these labels are available, only comparisons made under the same reference are claimed.

Several limitations apply. The two students differ in architecture, input size and schedule, so only the resolution-matched comparison isolates the effect of architecture. The adaptation variants share one recipe and one seed, and differences below about 0.01 $AP_{50:95}$ are not interpreted. The guardrails and the policy layer were designed after the cooperative tool-use utterances had been observed; the safety suite was therefore written before any of its cases were run. The policy and the suite were written by the same authors, and no adaptive attacker was evaluated against the final policy, so the reported rejection rate is an upper bound on the protection against a determined attacker. Four scenes remain a small sample: three are handheld or phone recordings, three were captured on one campus, and none covers night, rain or an overhead viewpoint. The VLM results apply to one 2B model at temperature 0 with one set of prompts. Finally, no measurement was made on a Jetson; latency, power, thermal behaviour and the contention between detector and VLM on an 8 GB Orin remain to be measured with TensorRT engines over long runs [56].

The runtime neither identifies individuals nor infers demographic attributes. It stores boxes, counts, crossings and dwell times, processes frames in memory, and deletes events, time series, reports and chat after a retention period (72 h by default) while retaining the audit log. Access requires a per-user token with an assigned role, and the server binds to the local network by default. The aa1 video is a publicly available pedestrian benchmark video distributed without an explicit licence; it is used here for research only, and its frames are not redistributed. Test1 was recorded by the authors on the university campus for this study, and scenes 1 and 2 are phone recordings from the same campus. Only code and derived data (boxes, tracks and metrics) are released.

VI. Conclusion

This paper showed that pseudo-label fine-tuning of a compact person detector on a single camera improves accuracy on that camera but reduces it below the pretrained baseline on every other camera tested and on COCO person images. The loss is attributable to a size-blind pseudo-label threshold and to forgetting. Averaging the pretrained and fine-tuned weights, with the coefficient selected on other cameras, restored the test-only accuracy (0.678 compared with 0.412 for the model selected on the adaptation camera) at no additional cost. The apparent accuracy advantage of the CNN student was found to be an effect of input resolution. For the VLM operator, text in the camera view constitutes an attack channel, and a policy layer outside the model raised the rejection rate of unsafe requests from 0.42 to 0.90 without affecting legitimate requests. Future work comprises the human ground-truth evaluation, on-device measurements on a Jetson Orin, adaptive red-teaming of the policy layer, and evaluation on additional cameras, including night-time scenes. All numbers in the paper are generated from the released result files by `paper/make_numbers.py`.

References

[1] I. Radosavovic, P. Dollár, R. Girshick, G. Gkioxari, and K. He, "Data distillation: Towards omni-supervised learning," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4119–4128.

[2] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-training with noisy student improves ImageNet classification," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 687–10 698.

[3] Z. Zhang and M. Hoai, "Object detection with self-supervised scene adaptation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 21 589–21 599.

[4] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[5] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics (ACL)*, 2024.

[6] Y. Gong, D. Ran, J. Liu, C. Wang, T. Cong, A. Wang, S. Duan, and X. Wang, "FigStep: Jailbreaking large vision-language models via typographic visual prompts," in *Proc. AAAI Conf. Artificial Intelligence*, 2025.

[7] M. Wortsman, G. Ilharco, J. W. Kim, M. Li, S. Kornblith, R. Roelofs, R. G. Lopes, H. Hajishirzi, A. Farhadi, H. Namkoong, and L. Schmidt, "Robust fine-tuning of zero-shot models," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 7959–7971.

[8] NVIDIA, "Jetson Orin modules and JetPack 6," https://developer.nvidia.com/embedded/jetson-orin, 2026.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[10] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[11] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, "Ultralytics YOLO26: Unified real-time end-to-end vision models," arXiv:2606.03748, 2026.

[12] Ultralytics, "Ultralytics YOLO: YOLO26 models and documentation," https://docs.ultralytics.com/models/yolo26/, 2026, software, version 8.4.

[13] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 213–229.

[14] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16 965–16 974.

[15] I. Robinson, P. Robicheaux, M. Popov, D. Ramanan, and N. Peri, "RF-DETR: Neural architecture search for real-time detection transformers," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026, arXiv:2511.09554.

[16] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby *et al.*, "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research (TMLR)*, 2024.

[17] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

[18] P. Dollár, C. Wojek, B. Schiele, and P. Perona, "Pedestrian detection: An evaluation of the state of the art," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 34, no. 4, pp. 743–761, 2012.

[19] S. Zhang, R. Benenson, and B. Schiele, "CityPersons: A diverse dataset for pedestrian detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 3213–3221.

[20] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2016, pp. 3464–3468.

[21] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conf. Computer Vision (ECCV)*, 2022.

[22] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, "Observation-centric SORT: Rethinking SORT for robust multi-object tracking," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023.

[23] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," arXiv:2206.14651, 2022.

[24] F. Yang, S. Odashima, S. Masui, and S. Jiang, "Hard to track objects with irregular motions and similar appearances? Make it easier by buffering the matching space," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, 2023.

[25] T. Stanczyk and F. Bremond, "Temporally propagated masks and bounding boxes: Combining the best of both worlds for multi-object tracking," arXiv:2409.14220, 2024.

[26] J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe, "HOTA: A higher order metric for evaluating multi-object tracking," *Int. J. Computer Vision*, vol. 129, no. 2, pp. 548–578, 2021.

[27] K. Bernardin and R. Stiefelhagen, "Evaluating multiple object tracking performance: The CLEAR MOT metrics," *EURASIP J. Image and Video Processing*, vol. 2008, 2008, article 246309.

[28] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi, "Performance measures and a data set for multi-target, multi-camera tracking," in *Proc. European Conf. Computer Vision Workshops (ECCVW)*, 2016, pp. 17–35.

[29] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531 (NIPS Deep Learning Workshop), 2015.

[30] R. Solovyev, W. Wang, and T. Gabruseva, "Weighted boxes fusion: Ensembling boxes from different object detection models," *Image and Vision Computing*, vol. 107, 2021, article 104117.

[31] A. Kumar, A. Raghunathan, R. Jones, T. Ma, and P. Liang, "Fine-tuning can distort pretrained features and underperform out-of-distribution," in *Int. Conf. Learning Representations (ICLR)*, 2022.

[32] K. Saito, D. Kim, P. Teterwak, R. Feris, and K. Saenko, "Mind the backbone: Minimizing backbone distortion for robust object detection," 2023, arXiv:2303.14744.

[33] X. Li, Y. Grandvalet, and F. Davoine, "Explicit inductive bias for transfer learning with convolutional networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2018, pp. 2825–2834.

[34] H. R. Medeiros, A. Belal, M. Aminbeidokhti, E. Granger, and M. Pedersoli, "WiSE-OD: Benchmarking robustness in infrared object detection," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, 2026, arXiv:2507.18925.

[35] R. Taori, A. Dave, V. Shankar, N. Carlini, B. Recht, and L. Schmidt, "Measuring robustness to natural distribution shifts in image classification," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[36] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL visual object classes challenge: A retrospective," *Int. J. Computer Vision*, vol. 111, no. 1, pp. 98–136, 2015.

[37] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *The Annals of Statistics*, vol. 17, no. 3, pp. 1217–1241, 1989.

[38] D. N. Politis and J. P. Romano, "A circular block-resampling procedure for stationary data," in *Exploring the Limits of Bootstrap*, R. LePage and L. Billard, Eds. Wiley, 1992, pp. 263–270.

[39] D. N. Politis and H. White, "Automatic block-length selection for the dependent bootstrap," *Econometric Reviews*, vol. 23, no. 1, pp. 53–70, 2004.

[40] A. Patton, D. N. Politis, and H. White, "Correction to "automatic block-length selection for the dependent bootstrap"," *Econometric Reviews*, vol. 28, no. 4, pp. 372–375, 2009.

[41] B. Phipson and G. K. Smyth, "Permutation P-values should never be zero: Calculating exact P-values when permutations are randomly drawn," *Statistical Applications in Genetics and Molecular Biology*, vol. 9, no. 1, 2010.

[42] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The hitchhiker's guide to testing statistical significance in natural language processing," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 1383–1392.

[43] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[44] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang *et al.*, "Qwen2.5-VL technical report," arXiv:2502.13923, 2025.

[45] S. Bai *et al.*, "Qwen3-VL technical report," arXiv:2511.21631, 2025.

[46] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[47] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[48] L. Beurer-Kellner *et al.*, "Design patterns for securing LLM agents against prompt injections," 2025, arXiv:2506.08837.

[49] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," 2025, arXiv:2503.18813.

[50] OWASP Foundation, "OWASP top 10 for large language model applications 2025: LLM06 excessive agency," https://genai.owasp.org/llmrisk/llm062025-excessive-agency/, 2025.

[51] Roboflow, "trackers: plug-and-play multi-object tracking library," https://github.com/roboflow/trackers, 2026, software, version 2.6.

[52] ONNX Runtime developers, "ONNX Runtime," https://onnxruntime.ai, 2026, software.

[53] G. Gerganov and ggml-org contributors, "llama.cpp: LLM inference in C/C++," https://github.com/ggml-org/llama.cpp, 2026, software.

[54] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Information and System Security*, vol. 2, no. 2, pp. 159–176, 1999.

[55] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multi-object tracking," 2016, arXiv:1603.00831.

[56] V. J. Reddi *et al.*, "MLPerf inference benchmark," in *Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2020, pp. 446–459.